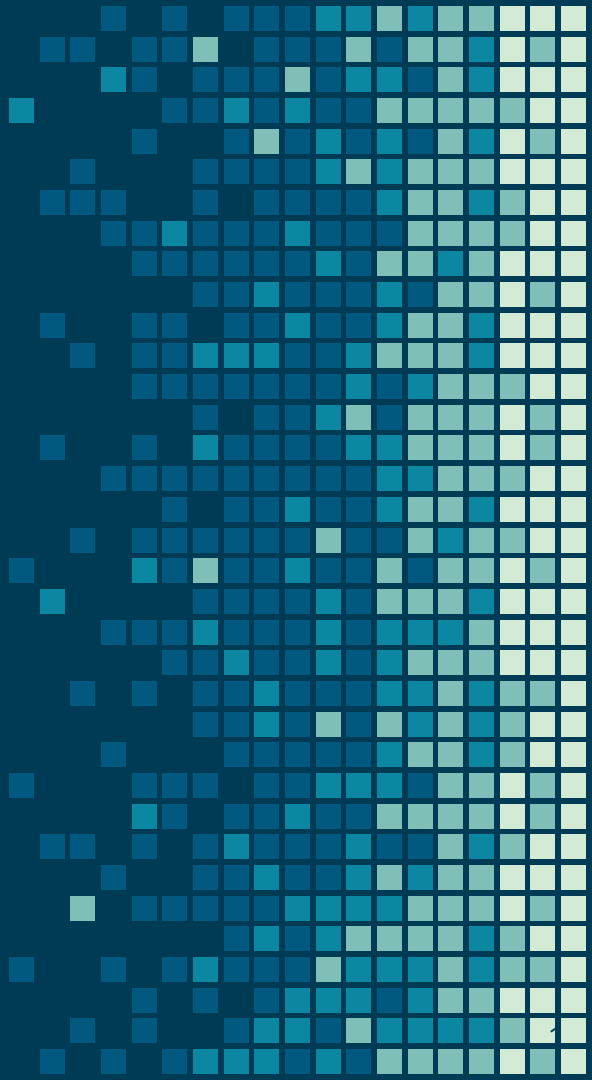
EMBEDDED CONTROL

Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



Inter-integrated Circuit (I^2C) communication bus

I²C basics (Philips late '80s)

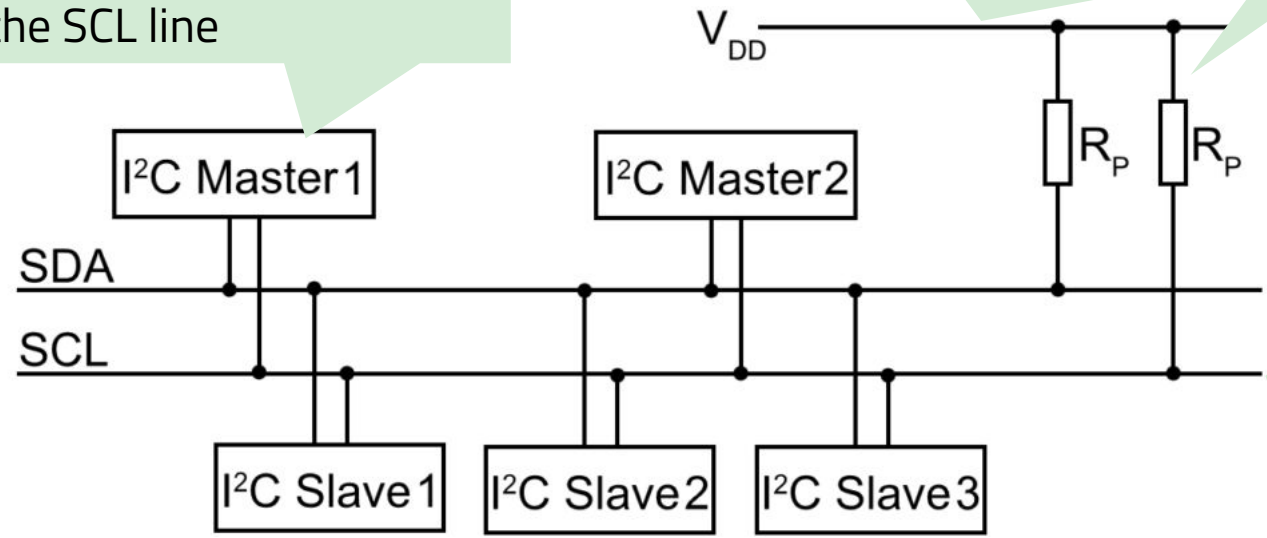
- **Serial bus** communication technology
- Two open-drain wires (wired AND logic!) are used:
  - Serial Clock Line (SCL): clock distributed to all nodes connected to the bus
  - Serial Data Line (SDA): data is sent clocked by the SCL over this line
- Devices connected to the bus are identified with an address (7-bits, 10-bits, or 16-bits)
- Devices can be both senders and receivers
- Master devices initiate transmission addressing a slave that should reply

I²C basics

The master initiates the transmission and then generates the Clock on the SCL line

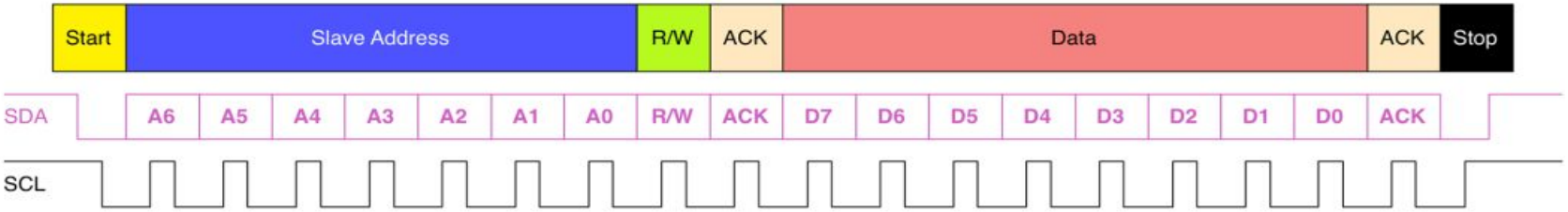
Lines are open drain: when the bus is idle no current in pull-up resistors => both lines are pulled high

Recommended value for pull-ups R_P=[1-4.7]kOhm



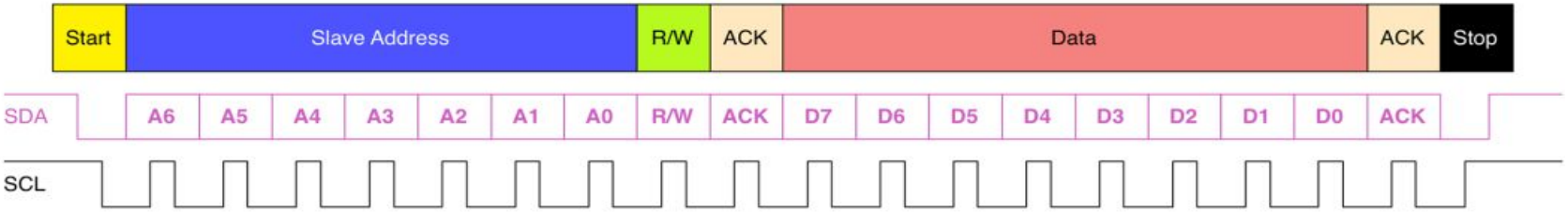
When the open-drain ports connected to SDA and SCL are driven they close the MOS and pull the line to GND

I²C Protocol basics



- Each communication is made of several 8-bit long data and must:
  - initiated with a **start bit**, finished with a **stop bit**
  - after each 8-bits are sent a bit is used to signal if the communication has been successful (ACK) or not (NACK)
- **Start condition**: Master pulls down SDA first (SCL is high), then starts to distribute the clock over the SCL line
- Stop condition: Master pulls up the SCL line and then releases the SDA line, i.e. pulls SDA high

I²C Protocol basics



- Master sends the address of the slave to communicate
- A R/W bit closes the address: if 0 master is sending if 1 master is requesting data
- Master releases the SDA line (high impedance) allowing the receiver to ACK or NACK
- If In the next clock pulse SDA is pulled low (by receiver), then the data frame is ACKed, otherwise we have a NACK
- An unspecified number of data bytes can be sent in the same way the address is sent (send byte, ack/nack, send byte, ack/nack,…,stop)

Five conditions for NACK

- No receiver is present on the bus with the transmitted address so there is no device to respond with an acknowledge (**most typical!**)
- The receiver is unable to receive or transmit because it is performing some real-time function and is not ready to start communication with the master
- During the transfer, the receiver gets data or commands that it does not understand
- During the transfer, the receiver cannot receive any more data
- A master-receiver must signal the end of the transfer to the slave transmitter

I2C speed

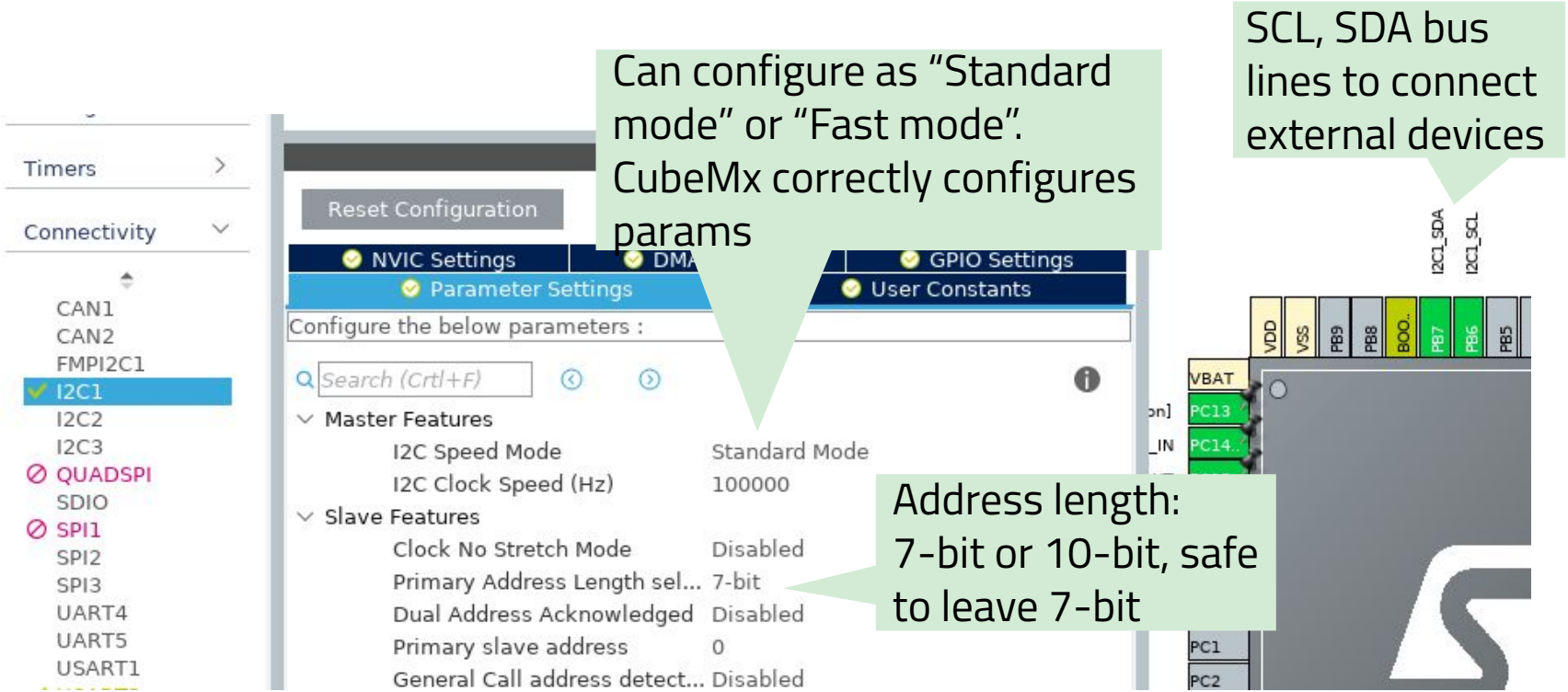
Typical I2C bitrates are the following:

- low-speed mode: 10 kbit/s
- standard mode: 100 kbit/s
- fast mode: 400 kbit/s
- high-speed mode: 3400 kbit/s

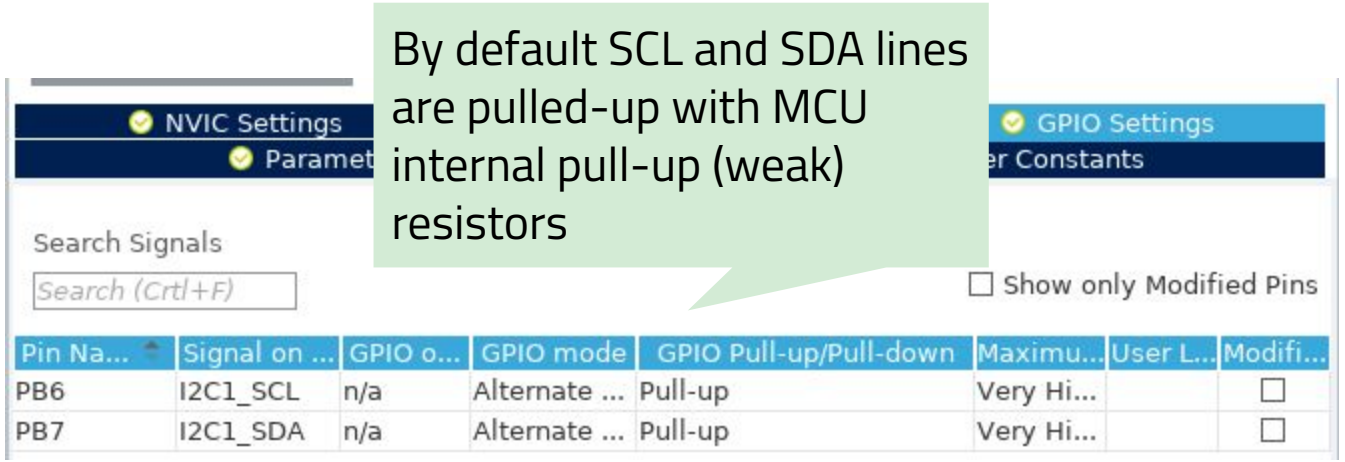
I2C HAL implementation

- All STM32 micros have at least an I2C peripheral
- Recall I2C is a bus, so you can connect many devices on the same two lines
- As for other peripherals the way to interact programmatically with I2C is through an handle, i.e., I2C_HandleTypeDef
- Very similar to the USART handler
- We will not see the specifics of the handler, if interested just search for `I2C_HandleTypeDef` typedef

I2C configuration with CubeMX



I2C configuration with CubeMX



- Weak internal pull-ups might cause issues in sampling bits (due to increased RC time constant)
- In such cases, SCL/SDA pins must be configured as open-drain and external pull-up resistors must be used

Using I2C programmatically with HAL

As all the other devices, I2C can be accessed in polling, interrupt and DMA mode

- Polling mode:
  ```
  HAL_I2C_Master_Receive(I2C_HandleTypeDef *hi2c,
  uint16_t DevAddress, uint8_t *pData, uint16_t Size,
  uint32_t Timeout);
  HAL_I2C_Master_Transmit(I2C_HandleTypeDef *hi2c,
  uint16_t DevAddress, uint8_t *pData, uint16_t Size,
  uint32_t Timeout)
  ```
- the only difference with `HAL_UART_Receive/Transmit` functions is that we need to specify the slave address

Using I2C programmatically with HAL

Interrupt mode:

- **HAL_I2C_Master_Transmit_IT**(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size);
- **HAL_I2C_Master_Receive_IT**(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size);
- As for the other peripherals, we first need to enable interrupts with CubeMX, and then implement callbacks, `HAL_I2C_MasterTxCpltCallback()`, `HAL_I2C_MasterRxCpltCallback()`

Typical use of I2C: memory addressable registers

Reading from an I2C slave:

- Master (MCU) **transmits** a "Command" to the slave requesting for a specific data, typically stored in a register on the slave
- F.i., the Master requests to read the gyroscope angular rate of x axis from an IMU sensor
- The slave receives the "Command" and prepares data to be read
- The master can then **read** the data

Example: reading from an I2C eeprom

```
/* compute the MSB and LSB parts of the memory address */
addr[0] = (uint8_t) ((MemAddress & 0xFF00) >> 8);
addr[1] = (uint8_t) (MemAddress & 0xFF);
/* MCU sends the memory location address where to start reading
data */
HAL_I2C_Master_Transmit(&hi2c, DevAddress, addr, 2,
HAL_MAX_DELAY);
/* Next we can retrieve the data from device */
HAL_I2C_Master_Receive(&hi2c, DevAddress, pData, len,
HAL_MAX_DELAY);
```

Make it easier using memory operations

- Many I2C slaves employ this communication pattern
- HAL provides a "standardized" way to access memory locations of I2C devices
- Write to a specific memory address of an I2C slave:
- `HAL_I2C_Mem_Write(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout);`
- `MemAddress`: memory address to write `pData` content
- MemAddSize: can either be `I2C_MEMADD_SIZE_8BIT, or I2C_MEMADD_SIZE_16BIT`
- Memory address length depends on the I2C slave: you must read the datasheet!

Memory read/write in interrupt mode

- Nothing different than the other devices
- `HAL_I2C_Mem_Write_IT`, `HAL_I2C_Mem_Read_IT` to write/read in interrupt mode

  ```
  HAL_I2C_Mem_Write_IT(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size)
  ```

- The callbacks to be implemented are:

  ```
  HAL_I2C_MemTxCpltCallback
  HAL_I2C_MemRxCpltCallback
  ```

Example configuring and reading from an IMU sensor

Example: read from MPU 6050 IMU

- Device address: `0x68`
- Notice: the address HAL wants considers an 8 bit address
- This means we need to **left shift of 1 position the `0x68`**
- Thus we will use `0xD0` (which is `0x68<<1`) as the address
- Register map here: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map1.pdf

PARAMETER	CONDITIONS	MIN	TYP	MAX	Units	Notes
SERIAL INTERFACE						
SPI Operating Frequency, All Registers Read/Write	MPU-6000 only, Low Speed Characterization		100 ±10%		kHz	
	MPU-6000 only, High Speed Characterization		1 ±10%		MHz	
SPI Operating Frequency, Sensor and Interrupt Registers Read Only	MPU-6000 only		20 ±10%		MHz	
I^2C Operating Frequency	All registers, Fast-mode			400	kHz	
	All registers, Standard-mode			100	kHz	
I^2C ADDRESS	AD0 = 0		1101000			
	AD0 = 1		1101001			

Reading the WHO_AM_I register

- Several IMUs provide a register to be read to be sure the device you are reading is the correct one
- The address on the IMU of the `WHO_AM_I` register is `0x75`
- Notice the addresses of the registers on the MPU6050 are 8 bit long
- To read the value of the register we can use the `HAL_I2C_Mem_Read` function:

```
HAL_I2C_Mem_Read(&hi2c1, 0xD0, 0x75,
                 I2C_MEMADD_SIZE_8BIT, &whoami, 1, 100);
```

Configuring the sensor (1/3)

- There are a number of registers to be configured to properly set up this specific IMU
- Looking at the datasheet we need to set: `PWR_MGMT_1` to `0x00` to wake up the sensor

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5		Bit2	Bit1	Bit0
6B	107	DEVICE _RESET	SLEEP	CY		CLKSEL[2:0]		

If equal to 1 the sensor sleeps

Configuring the sensor (2/3)

- The `SMPRT_DIV`, the sample rate divider

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
19	25	SMPLRT_DIV[7:0]							

It specifies the divider from the gyroscope output rate used to generate the Sample Rate for the MPU-60X0:

```
Sample rate = Gyro rate/(1+SMPRT_DIV)
```

- Since the Gyro rate by default is 8kHz to set the sample rate to 1 kHz we set `SMPRT_DIV` to 7 (`0x07`)

Configuring the sensor (3/3)

- Gyroscope and accelerometer are configured using `ACCEL_CONFIG` and `GYRO_CONFIG` (the registers have the same structure)
- `GYRO_CONFIG`

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
1B	27	XG_ST	YG_ST	ZG_ST	FS_SEL[1:0]		-	-	-

- `XG_ST`, `YG_ST`, `ZG_ST` if set to 1 put the corresponding axis in self test. Since we do not need to test the gyro we set those bits to `0`
- `FS_SEL` sets the gyro full scale range
- We set the full range scale to +/- 250 deg/s
- Thus, the value of `GYRO_CONFIG` is `0`

FS_SEL	Full Scale Range
0	± 250 °/s
1	± 500 °/s
2	± 1000 °/s
3	± 2000 °/s

Gyroscope full scale range

AFS_SEL	Full Scale Range
0	± 2*g*
1	± 4*g*
2	± 8*g*
3	± 16*g*

Accelerom. full scale range

IMU Configuration summary

Register	Address	Value	Note
PWR_MGMNT_1	0x6B	0x00	to power up sensor
SMPRT_DIV	0x19	0x07	set sampling rate to 1kHz
GYRO_CONFIG	0x1B	0x00	full scale +/- 250 deg/s
ACCEL_CONFIG	0x1C	0x00	full scale +/- 2g

Reading angular rates and accelerations

Datasheet's registers table:

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT[7:0]							

To read data from the sensor is sufficient to read the correct bytes addressed in this table

Reading all accelerometers axes

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT[7:0]							

We will read 6 bytes starting from address 0x3B:

```
uint8_t buffer[6];
HAL_I2C_Mem_Read(&hi2c1,0xD0,0x3B,I2C_MEMADD_SIZE_8BIT,buffer,6,100);
```